



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



Corso di Elementi di Intelligenza Artificiale

Confronto di algoritmi di ricerca ed euristiche per il pathfinding

Anno Accademico 2025/2026

DOCENTE: Sperlì Giancarlo

Manco Matteo - N46007842

Marrazzo Silverio - N46007685

Rumieri Ginevra - N46007644

Sommario

1.1 Obiettivo del progetto	3
1.2 Formalizzazione PEAS.....	3
1.3 Classificazione dell'ambiente	4
1.4 Problema single-state e ricerca offline.....	4
2. Dataset e Modellazione a Grafo.....	5
2.1 Sorgente dei dati: OpenStreetMap	5
2.2 Download e gestione del grafo: OSMnx.....	5
2.3 Modellazione a grafo	5
2.4 Scelta dei metri rispetto al tempo	5
2.5 Problema di ricerca	5
3. Metodologie Tecniche	6
3.1 Motore di ricerca unificato	6
3.2 A* e Greedy Best-First	6
3.3 Breadth-First Search (BFS)	6
3.4 Euristiche a confronto	6
3.4.1 Euristiche Zero	7
3.4.2 Euristiche Euclidee	7
3.4.3 Euristiche Landmark (ALT)	7
3.4.4 Euristiche Pesate	7
3.5 Fattore di ramificazione effettivo b^*	8
3.6 Architettura del sistema e modalità di esecuzione	8
4. Discussione dei Risultati	9
4.1 Setup sperimentale.....	9
4.2 Metriche di valutazione	9
4.3 Risultati aggregati	9
4.4 Verifica dell'ottimalità e analisi dell'errore	10
4.5 Fattore di ramificazione	11
4.6 Nodi espansi, tempo di esecuzione	11
4.7 Scala logaritmica	12
4.9 Validazione cartografica dei percorsi per classi di distanza.....	12
5. Conclusioni	16

1. Introduzione

1.1 Obiettivo del progetto

Il progetto ha l'obiettivo di sviluppare e documentare un framework di benchmarking automatico per l'analisi quantitativa e il confronto prestazionale di diversi algoritmi di ricerca nello spazio degli stati, applicati al problema del routing su reti stradali reali. Come scenario sperimentale di riferimento abbiamo selezionato la rete viaria della città metropolitana di Napoli.

A differenza di un navigatore interattivo tradizionale, il sistema è strutturato come un framework di test automatizzato progettato per:

- Effettuare un'analisi statistica e riproducibile. Il programma campiona automaticamente coppie di nodi origine-destinazione distribuite su diverse fasce di distanza.
- Raccogliere metriche computazionali. Per ogni ricerca vengono infatti registrati i nodi espansi, il picco della frontiera in memoria e il fattore di ramificazione effettivo, oltre al tempo di calcolo e all'accuratezza del costo rispetto all'ottimo C^* .
- Tradurre i dati quantitativi raccolti in rappresentazioni grafiche e cartografiche di sintesi, facilitando l'analisi comparativa tra approcci non informati, euristiche e tecniche basate su landmark.

1.2 Formalizzazione PEAS

Per progettare il nostro navigatore come agente razionale, abbiamo analizzato il contesto operativo secondo il modello **PEAS**. Questo schema ci ha permesso di isolare gli obiettivi del sistema e gli strumenti che ha a disposizione per muoversi sulla mappa.

- **Performance measure:** il successo dell'agente viene valutato in base a criteri di efficienza e accuratezza. L'obiettivo principale è la minimizzazione della lunghezza del percorso, seguita dalla riduzione del tempo di calcolo e del numero di nodi espansi nella frontiera. Il sistema deve inoltre garantire l'ottimalità del costo totale C^* per gli algoritmi che lo prevedono e assicurare la massima chiarezza nella visualizzazione grafica del percorso finale.
- **Environment:** il nostro agente si muove sulla rete stradale reale, che abbiamo modellato come un grafo pesato e orientato, scaricato tramite la libreria OSMnx e salvato localmente in formato *.graphml*. La memorizzazione locale evita continui download durante l'esecuzione e garantisce la stabilità dei dati durante i test.
- **Actuators:** gli strumenti attraverso cui l'agente modifica lo stato del sistema o comunica i risultati comprendono la selezione logica del prossimo nodo da espandere durante la ricerca, la scrittura su disco dei risultati (log e dati delle medie), la generazione del pannello di visualizzazione delle metriche e il salvataggio dei file grafici contenenti i percorsi tracciati sul grafo stradale.
- **Sensors:** per conoscere l'ambiente circostante, il sistema legge la struttura del grafo precaricato, (acquisendo gli identificativi dei nodi, le loro coordinate GPS e la lunghezza esatta degli archi) e i parametri statici definiti nel file di configurazione, come il SEED di generazione pseudocasuale e gli intervalli di distanza per il campionamento dei nodi origine-destinazione.

1.3 Classificazione dell'ambiente

L'ambiente in cui opera l'agente può essere classificato come:

- **Completamente osservabile:** l'intero grafo stradale viene caricato interamente in memoria prima di avviare la ricerca. L'agente ha quindi una visibilità totale sullo stato del sistema.
- **Deterministico:** ad azione corrisponde una sola conseguenza, senza componenti stocastiche legate ad esempio a condizioni variabili di traffico.
- **Statico:** il grafo non subisce modifiche durante la fase di calcolo e pianificazione.
- **Discreto:** lo spazio degli stati (gli incroci/nodi) e l'insieme delle azioni (le strade/archi) sono finiti ed enumerabili.
- **Sequenziale:** le decisioni prese ad ogni passo dell'algoritmo (quale nodo espandere) dipendono da quelle precedenti tramite il costo accumulato e condizionano le espansioni successive.
- **Single-Agent:** il processo di pianificazione è interamente a carico di un unico agente, senza la presenza di entità esterne che possano collaborare per la scelta del percorso.

1.4 Problema single-state e ricerca offline

Date le proprietà di completa osservabilità e determinismo dell'ambiente stradale modellato, il compito di pianificazione è classificabile come un *single-state problem*: l'agente, infatti, è in grado di calcolare con certezza lo stato in cui si trova in ogni istante e l'esito deterministico di ogni sequenza di azioni, per cui non è necessario alcun meccanismo di percezione dinamica durante l'elaborazione.

Di conseguenza, la ricerca viene condotta interamente **offline**: l'algoritmo esplora e valuta l'albero di ricerca basandosi esclusivamente sulla rappresentazione logica della mappa memorizzata, calcolando l'intero percorso ottimale prima che avvenga la fase di esportazione dei risultati e la produzione dei relativi grafici e tracciati cartografici. Questo permette di separare nettamente la fase di risoluzione algoritmica (ossia il focus di questo progetto) dalla successiva fase di analisi statistica e renderizzazione del tragitto.

2. Dataset e Modellazione a Grafo

2.1 Sorgente dei dati: OpenStreetMap

Per ottenere la rete stradale su cui far muovere l'agente abbiamo utilizzato *OpenStreetMap (OSM)*, un progetto open-source che fornisce dati geografici liberi. OSM rappresenta la rete come un insieme di nodi (punti geografici nello spazio, che nel nostro contesto rappresentano gli incroci stradali) e way (sequenze di nodi che formano i segmenti stradali veri e propri). Ogni elemento è arricchito da metadati fondamentali per il routing, come il senso di marcia e la tipologia di strada.

2.2 Download e gestione del grafo: OSMnx

Il compito di scaricare e modellare questi dati è affidato a *OSMnx*, una libreria Python che interroga i server di OpenStreetMap per l'area geografica specifica nei parametri di configurazione del benchmark e converte la mappa in un grafo orientato, in cui ogni nodo è associato alle proprie coordinate geografiche e ogni arco riporta la lunghezza reale del rispettivo segmento stradale espressa in metri.

Per evitare download ripetuti, il grafo scaricato viene salvato localmente in formato `.graphml` e ricaricato direttamente nelle esecuzioni successive, azzerando i tempi di attesa del download.

2.3 Modellazione a grafo

La struttura della rete stradale si presta a essere formalizzata secondo i paradigmi della ricerca nello spazio degli stati: ogni nodo del grafo costituisce uno **stato** del problema, ogni arco un'**azione** disponibile il cui costo coincide con il peso già introdotto precedentemente, e la funzione successore è definita implicitamente dalla struttura di adiacenza del grafo stesso. Lo **stato iniziale** e lo **stato obiettivo** sono definiti da una coppia di nodi del grafo, campionata dal sistema per ciascuna istanza sperimentale.

È importante notare che il concetto di nodo del grafo non è identico al nodo inteso come struttura dati dell'algoritmo di ricerca, ovvero il costrutto che affianca allo stato il predecessore, l'azione che lo ha generato e il costo del cammino accumulato, le quali sono informazioni utilizzate dall'algoritmo, non dal problema in sé.

2.4 Scelta dei metri rispetto al tempo

Il peso di ciascun arco è espresso in metri anziché in tempo di percorrenza stimato, poiché l'attributo di velocità massima su OSM è incompleto e disomogeneo, e la velocità dipende comunque da fattori non modellati nel sistema, come il traffico o gli orari. I metri sono inoltre l'unità naturale per confrontare il costo $g(x)$ con le euristiche geometriche che abbiamo utilizzato.

2.5 Problema di ricerca

Applicando la formalizzazione discussa nel capitolo precedente al nostro caso di studio, si ottiene:

- **Stato iniziale:** nodo del grafo stradale estratto casualmente dal sistema per definire il punto di partenza di una specifica simulazione.
- **Azioni:** per ogni nodo n , l'insieme degli archi uscenti verso i nodi adiacenti.
- **Funzione successore:** restituisce l'esito di ogni azione, ovvero il nodo raggiunto percorrendo l'arco e il relativo costo.
- **Goal test:** condizione di arresto che verifica se il nodo corrente coincide con il nodo obiettivo estratto dal sistema per quella simulazione.
- **Costo del cammino:** somma dei pesi degli archi attraversati dallo stato iniziale allo stato obiettivo.

3. Metodologie Tecniche

3.1 Motore di ricerca unificato

A differenza di un'implementazione con funzioni separate per ciascun algoritmo, abbiamo adottato un motore di ricerca comune da cui tutte le strategie di ricerca informata vengono derivate come semplici funzioni di facciata. Le quattro varianti di A* (Dijkstra, Euclidea, Landmark, Pesata) e Greedy Best-First sono realizzate come semplici funzioni di facciata che si appoggiano a questo nucleo in comune.

Le diverse strategie si differenziano esclusivamente per la funzione euristica fornita e per l'attivazione o meno di un parametro di controllo booleano. Quando questo parametro è abilitato, il motore valuta gli stati combinando il costo effettivo accumulato dal punto di partenza con la stima residua verso l'obiettivo, realizzando la logica di A*. Quando invece il parametro è disattivato, la componente del costo reale viene interamente ignorata e la coda di priorità viene ordinata sulla base della sola stima euristica, istanziando l'algoritmo Greedy Best-First.

Questa unificazione architetturale garantisce che tutte le varianti di A* e Greedy Best-First condividano gli stessi meccanismi di gestione della frontiera e delle metriche di conteggio, fatta ad eccezione della BFS, implementata separatamente con una frontiera FIFO, coerentemente con la sua natura di algoritmo non informato usato come termine di paragone

3.2 A* e Greedy Best-First

Le quattro varianti della ricerca A* combinano il costo reale finora sostenuto con la stima del costo residuo per raggiungere la destinazione. Al contrario, l'algoritmo Greedy Best-First si basa esclusivamente sulla stima euristica, escludendo del tutto dalla valutazione il costo già accumulato dal punto di partenza. Questo rende la ricerca Greedy molto rapida e mirata nel dirigersi verso l'obiettivo, sacrificando l'ottimalità. Infatti, non considerando la strada già percorsa, l'algoritmo può facilmente preferire un cammino che appare temporaneamente vantaggioso in base alla sola stima aerea residua, ma che alla fine si rivela più lungo di un'alternativa scartata nelle prime fasi perché ritenuta meno promettente dall'euristica.

3.3 Breadth-First Search (BFS)

Come termine di paragone non informato, abbiamo implementato la Breadth-First Search, che esplora il grafo per livelli crescenti senza alcuna informazione euristica. La BFS individua il cammino con il minor numero di archi: su un grafo pesato come quello stradale, un percorso con pochi incroci ma segmenti molti lunghi viene preferito a un percorso più breve in metri ma con più incroci intermedi. In termini di complessità, BFS mantiene in frontiera l'intero livello corrente, con un fabbisogno di memoria $O(b^d)$ analogo a quello di Dijkstra, poiché nessuna delle due strategie sfrutta informazione euristica per orientare l'esplorazione.

3.4 Euristiche a confronto

Il calcolo del costo stimato verso l'obiettivo viene delegato a un modulo dedicato, che utilizza un approccio basato su Factory e Closure. Questo approccio ci ha permesso di isolare le operazioni più pesanti, come la creazione di una mappatura interna per l'accesso immediato alle coordinate geografiche o il calcolo preventivo delle distanze per l'euristica Landmark, eseguendole una sola volta all'avvio del programma. I risultati di questo preprocessing vengono poi inseriti all'interno della funzione euristica finale, la quale rimane l'unica ad essere richiamata nel ciclo operativo di A*.

In questo modo l'algoritmo non deve più consultare la struttura del grafo a runtime, ma accede istantaneamente a dati già pronti in memoria, riducendo i tempi di risposta.

3.4.1 Euristica Zero

Questa configurazione restituisce un costo stimato costantemente pari a zero per qualsiasi stato. Di conseguenza, la valutazione complessiva si riduce al solo costo reale accumulato dal punto di partenza, facendo regredire il motore di ricerca informata all'algoritmo di Dijkstra. Essendo una stima nulla, risulta banalmente ammissibile e consistente. Siccome priva di qualsiasi informazione direzionale per guidare la frontiera, funge da baseline non informata per quantificare l'efficacia reale delle euristiche successive.

3.4.2 Euristica Euclidea

Questa variante si basa sul calcolo della distanza geometrica in linea d'aria tra il nodo corrente e la destinazione. Per ottenere una misurazione accurata su una mappa reale, il sistema applica una proiezione equirettangolare locale, convertendo i gradi di latitudine e longitudine in metri effettivi sulla base delle coordinate geografiche dell'area urbana configurata nel benchmark.

Dal punto di vista teorico, la distanza in linea d'aria rappresenta la soluzione esatta del problema rilassato in cui si cancella il vincolo di dover seguire la rete viaria. Risulta essere matematicamente ammissibile e consistente, poiché il cammino stradale effettivo tra due punti non potrà mai essere inferiore alla retta geometrica che li unisce. Tuttavia, all'interno di un tessuto cittadino, questa stima si rivela strutturalmente debole: a causa dell'andamento reale delle strade (caratterizzato da curve, svolte obbligate e sensi unici), essa tende a sottostimare sensibilmente la distanza di cammino reale, coprendo in media solo una frazione del costo effettivo.

3.4.3 Euristica Landmark (ALT)

Si basa sulla tecnica ALT (A^* , Landmarks, Triangle inequality), la quale sfrutta un insieme ridotto di nodi di riferimento, denominati landmark, per i quali vengono pre-calcolate offline le distanze esatte di cammino minimo verso e da tutti gli altri nodi della rete.

Per ottimizzare il loro posizionamento ed evitare scelte arbitrarie, i landmark vengono selezionati tramite una strategia greedy maxmin (farthest): partendo da un nodo iniziale casuale, viene individuato il punto più distante tramite l'algoritmo di Dijkstra, per poi aggiungere progressivamente i landmark successivi massimizzando la distanza minima da quelli già selezionati.

Poiché la rete stradale urbana è rappresentata da un grafo orientato caratterizzato da numerosi sensi unici, la distanza stradale tra due nodi non è simmetrica. Di conseguenza, il sistema esegue i calcoli preliminari sia sul grafo normale (distanze in uscita dal landmark) sia sul grafo trasposto (distanze in entrata). Il massimo tra queste due stime distinte basate sulla disuguaglianza triangolare, garantisce un'euristica ammissibile e consistente, evitando sovrastime.

Per contenere i costi di calcolo a runtime ed evitare di calcolare la disuguaglianza triangolare su tutti i landmark disponibili a ogni iterazione, prima di avviare ciascuna ricerca viene eseguita una preselezione dinamica dei landmark attivi, isolando un solo sottoinsieme di soli $N_{ATTIVI} = 3$ landmark che offre la stima più stringente per la specifica coppia di origine e destinazione.

3.4.4 Euristica Pesata

Questa variante introduce una modifica alla funzione di valutazione classica di A^* impostando un peso pari a $w = 2$. Moltiplicando il valore dell'euristica per un fattore superiore a uno, l'algoritmo assegna molta più importanza alla stima geometrica rispetto al costo reale accumulato dal punto di partenza. Questo comportamento riduce l'esplorazione laterale dei nodi, focalizzando la ricerca quasi esclusivamente in direzione del traguardo e velocizzando l'esecuzione. Tuttavia questa accelerazione comporta un trade-off: raddoppiando il valore dell'euristica, quest'ultima tende a sovrastimare il costo residuo effettivo, perdendo la proprietà di ammissibilità. Questo significa che il Weighted A^* non può garantirci matematicamente di trovare il percorso più breve in assoluto.

3.5 Fattore di ramificazione effettivo b^*

Per quantificare sinteticamente l'efficacia di una strategia di ricerca, calcoliamo il fattore di ramificazione effettivo b^* , definito dall'equazione della somma:

$$1 + b^* + (b^*)^2 + \dots + (b^*)^d = N + 1$$

dove N è il numero di nodi espansi e d la profondità del cammino trovato. Non essendo un'equazione risolvibile in forma chiusa, viene risolta numericamente tramite metodo di bisezione. Per garantire la stabilità del metodo anche su percorsi lunghi, dove l'elevamento a potenza di d potrebbe causare overflow durante la ricerca, l'estremo superiore dell'intervallo di bisezione non è fissato a priori, ma calcolato dinamicamente, garantendo la convergenza dell'algoritmo indipendentemente dalla scala del problema.

3.6 Architettura del sistema e modalità di esecuzione

Il file principale, `driver.py`, opera come un benchmark batch automatico, progettato per raccogliere dati statistici su un numero elevato di coppie origine-destinazione anziché validare un singolo percorso scelto manualmente. Innanzitutto, il sistema verifica la presenza in locale, in formato `.graphml` nella cartella `mappe/`, del grafo stradale relativo all'area configurata. In assenza del file, il grafo viene scaricato tramite OSMnx e salvato automaticamente per velocizzare le esecuzioni successive.

Successivamente viene estratta la componente fortemente connessa più grande del grafo: poiché in un grafo orientato l'esistenza di un arco da un punto ad un altro non implica l'esistenza del percorso inverso, restringere l'analisi alla componente fortemente connessa garantisce che ogni coppia di nodi campionata sia mutuamente raggiungibile, azzerando i fallimenti di routing dovuti a nodi isolati o raggiungibili in una sola direzione.

In seguito, il sistema esegue un campionamento statistico di coppie di nodi tramite un generatore pseudocasuale a SEED fisso (pari a 42, per garantire la piena riproducibilità dei risultati), suddividendo le coppie campionate in classi di distanza in linea d'aria crescente, da 0.5km fino a 50km.

Infine, per ciascuna coppia valida vengono eseguiti in sequenza tutti e sei gli algoritmi registrati sullo stesso problema, garantendo la parità di condizioni sperimentali e registrando le rispettive metriche di performance. Al termine del benchmark, il sistema esporta automaticamente nella cartella `risultati/` un file `risultati.csv` con i dati grezzi di ogni singolo test, un pannello grafico `confronto_distanze.png`, che riporta la mediana e la banda interquartile delle metriche principali al crescere della distanza, e le immagini del percorso ottimo calcolato per una coppia rappresentativa di ciascuna fascia di distanza.

4. Discussione dei Risultati

4.1 Setup sperimentale

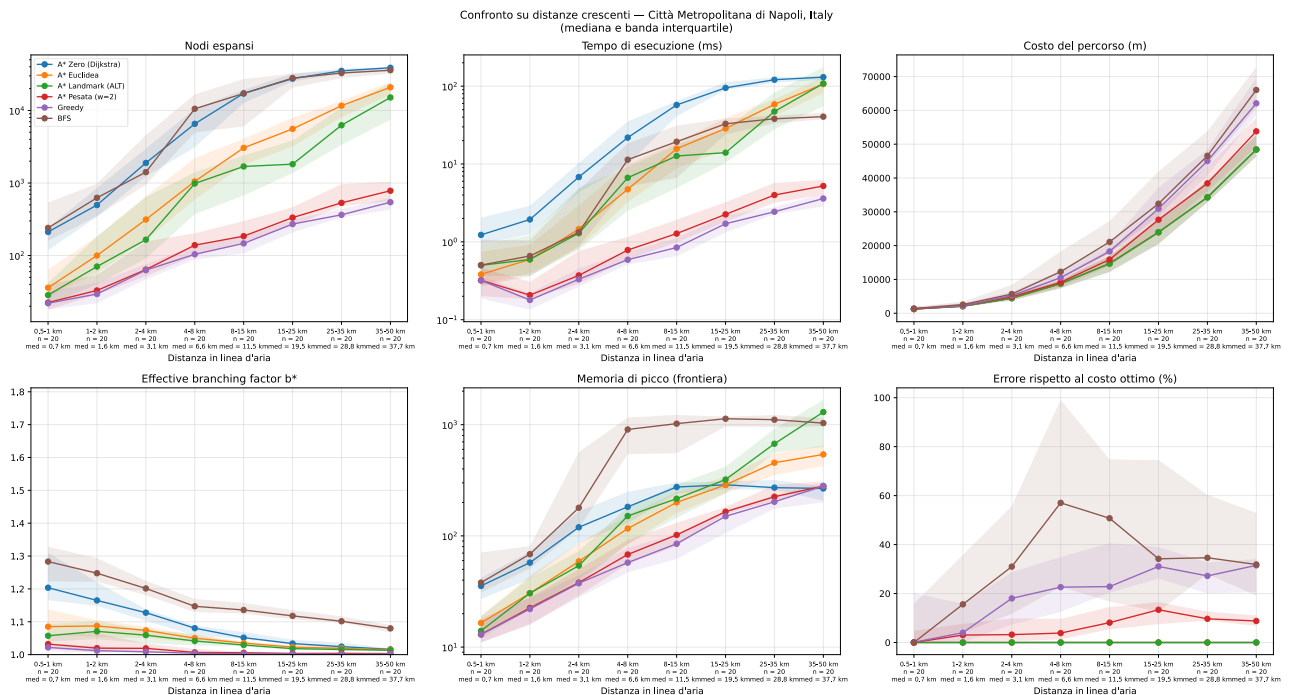
Come già anticipato nei capitoli precedenti, la valutazione sperimentale è stata condotta con l'esecuzione reale del benchmark batch (driver.py) sul grafo stradale della città metropolitana di Napoli (40.574 nodi, 87.126 archi), con pre-processing dell'euristica ALT su 8 landmark. Dopo l'estrazione della componente fortemente connessa, il sistema ha campionato 20 coppie origine-destinazione per ciascuna di 8 classi di distanza in linea d'aria crescente (da 0,5-1 km fino a 35-50 km), per un totale di 160 coppie e 960 esecuzioni (160 coppie $\times$ 6 algoritmi: Zero/Dijkstra, Euclidea, Landmark/ALT, Pesata $w=2$, Greedy, BFS) salvate in risultati.csv. La seguente analisi si basa sull'intero campione, riportando in mediana e banda interquartile (IQR) per ciascuna metrica, la validazione statistica su larga scala.

4.2 Metriche di valutazione

Il confronto si basa sulle metriche già introdotte (costo ed errore % rispetto a C^* , archi, nodi espansi, tempo, picco frontiera, b^*), qui aggregate su ciascuna delle 20 coppie per bin tramite mediana, la statistica meno sensibile a eventuali outlier rispetto alla media, coerentemente con quanto riportato nel pannello grafico del confronto delle distanze.

4.3 Risultati aggregati

L'immagine riporta il pannello completo generato dal benchmark: mediana (linea) e banda interquartile (area ombreggiata) di nodi espansi, tempo di esecuzione, costo del percorso, fattore di ramificazione effettivo b^* e picco di memoria della frontiera, in funzione della distanza in linea d'aria fra le coppie testate.



Le seguenti tabelle riportano rispettivamente i valori mediani esatti di nodi espansi e tempo di esecuzione per ciascun algoritmo, su tutti gli 8 pin di distanza.

Algoritmo	0,5-1	1-2	2-4	4-8	8-15	15-25	25-35	35-50
A* Zero (Dijkstra)	212	496	1.892	6.534	17.197	27.426	35.185	38.824
A* Euclidea	36	100	314	1.052	3.053	5.566	11.630	20.856
A* Landmark (ALT)	28	70	166	984	1.691	1.818	6.256	15.098
A* Pesata (w = 2)	22	33	64	139	185	334	532	782
Greedy	22	30	63	104	147	272	364	544
BFS	240	625	1.412	10.552	17.132	27.942	32.831	35.842

Algoritmo	0,5-1	1-2	2-4	4-8	8-15	15-25	25-35	35-50
A* Zero (Dijkstra)	1,23	1,94	6,80	21,84	57,36	95,03	121,01	130,68
A* Euclidea	0,38	0,60	1,46	4,72	15,70	28,65	58,40	106,41
A* Landmark (ALT)	0,50	0,59	1,29	6,66	12,68	14,01	47,24	108,45
A* Pesata (w = 2)	0,32	0,21	0,37	0,78	1,28	2,25	4,00	5,22
Greedy	0,32	0,18	0,33	0,59	0,85	1,71	2,44	3,60
BFS	0,50	0,66	1,33	11,36	19,35	32,90	38,13	40,62

4.4 Verifica dell'ottimalità e analisi dell'errore

Su tutte le 480 esecuzioni relative alle tre euristiche ammissibili (Zero, Euclidea, Landmark: 160×3 configurazioni), l'errore percentuale rispetto al costo ottimo C^* è risultato esattamente 0,00%. Non abbiamo riscontrato nessuna eccezione sull'intero campione, né nelle medie né nei valori esterni: questo dimostra come l'algoritmo non abbia mai fallito nel trovare il cammino minimo reale in metri.

Per le strategie non garantite, invece, abbiamo osservato un errore che cresce inizialmente con la distanza fino a stabilizzarsi su un altopiano. In particolare, la Pesata si è assestata su uno scostamento compreso tra circa l'8% e il 13% oltre gli 8 km, mentre la ricerca Greedy ha mostrato una precisione inferiore, con errori tra circa il 23% e il 31%. Pesato resta comunque sistematicamente più vicino a C^* rispetto a Greedy su ogni bin di distanza.

L'errore di BFS cresce da un valore quasi nullo sotto 1 km fino a un massimo di circa il 57% nel bin 4-8 km, per poi diminuire e stabilizzarsi intorno al 32% nel bin più esteso (35-50 km).

4.5 Fattore di ramificazione

I risultati dei benchmark confermano la gerarchia di efficacia teorica delle strategie analizzate: il fattore di ramificazione effettivo b^* si mantiene sistematicamente più basso per l'euristica Landmark e per A^* Pesato rispetto all'euristica Euclidea, a Dijkstra e alla ricerca in ampiezza (BFS) all'interno del medesimo intervallo di distanza. Tale evidenza rispecchia direttamente il maggior grado di informazione incorporato nelle rispettive funzioni di valutazione.

Tuttavia, l'analisi dell'andamento mostra che il valore di b^* tende a decrescere per tutte le strategie al crescere della distanza tra i nodi, convergendo verso il valore ideale pari a 1 anche per gli algoritmi meno informati. Ad esempio, l'algoritmo di Dijkstra registra un b^* medio che passa da 1,204 nella fascia di distanza più breve a 1,016 in quella più estesa, mentre la BFS scende da 1,283 a 1,080.

Questo comportamento evidenzia come il fattore di ramificazione effettivo non sia una metrica indipendente dalla scala del problema. La sua stessa definizione matematica, espressa dall'equazione della somma geometrica dei nodi espansi rispetto alla profondità del cammino, implica che a parità di efficienza relativa dell'algoritmo, un percorso caratterizzato da una profondità (numero di archi) sensibilmente maggiore richieda un valore di b^* matematicamente più vicino all'unità per giustificare il medesimo ordine di grandezza dei nodi totali esplorati.

Di conseguenza, sebbene il confronto basato su b^* rimanga pienamente valido se effettuato tra algoritmi diversi all'interno dello stesso intervallo di distanza (dove la profondità dei cammini è comparabile), risulta metodologicamente fuorviante confrontare direttamente i valori di b^* ottenuti su percorsi brevi con quelli registrati su tratte molto lunghe, o tra reti stradali di scale e dimensioni differenti, siccome la sola profondità della soluzione andrebbe a mascherare la reale efficienza dell'algoritmo.

4.6 Nodi espansi, tempo di esecuzione

I dati relativi al volume di esplorazione evidenziano come l'euristica Landmark riesca a espandere il minor numero di nodi tra tutte le varianti ammissibili in ognuna delle fasce di distanza esaminate. Già nel primo intervallo (0,5–1 km), la tecnica basata su landmark richiede l'espansione di una mediana di 28 nodi contro i 36 dell'approccio geometrico euclideo. Questo divario si amplia progressivamente all'aumentare delle distanze, raggiungendo il picco nella fascia tra i 15 e i 25 km, dove l'euristica Euclidea espande un numero di nodi circa 3 volte superiore rispetto a Landmark (5 566 nodi contro i 1 818 di quest'ultima), per poi ridursi sulle tratte più lunghe.

L'analisi dei tempi di esecuzione delinea tuttavia un quadro differente. L'euristica Landmark risulta la più rapida sulla maggior parte delle fasce, e in modo netto e consistente nella banda intermedio-lunga (tra gli 8 e i 25 km); sui percorsi più brevi e su quelli molto estesi le due euristiche si equivalgono sostanzialmente, con l'euristica Euclidea che occasionalmente prevale sul primo bin (0,5–1 km) e sul più esteso (35-50 km).

Questo comportamento trova una spiegazione nel bilanciamento tra complessità per nodo e selettività della ricerca. L'euristica Landmark presenta un costo computazionale per singolo nodo intrinsecamente più elevato rispetto alla distanza euclidea, poiché richiede l'accesso alle distanze pre-calcolate dei tre landmark selezionati dinamicamente come attivi, a fronte del singolo calcolo geometrico in linea d'aria

richiesto dalla formula euclidea. Sulle brevi distanze, dove il numero assoluto di nodi esaminati è ridotto, l'impatto di questo sovraccarico per nodo può superare i benefici della contrazione dell'albero di ricerca. Al contrario, sulle distanze intermedie e lunghe la riduzione dei nodi espansi operata da Landmark compensa ampiamente il costo computazionale della singola valutazione, ottimizzando il tempo complessivo di CPU.

Infine, le configurazioni di A* Pesato e Greedy si confermano come i sistemi più rapidi in assoluto in ogni scenario testato. La strategia Greedy registra i tempi di elaborazione più bassi grazie all'esclusione del costo accumulato $g(n)$ dalla propria funzione di valutazione, compensando però tale rapidità con un errore percentuale sul costo del cammino che risulta sistematicamente da due a più di cinque volte superiore rispetto a quello registrato dalla variante pesata.

4.7 Scala logaritmica

Per i nodi espansi, i tempi di esecuzione e il picco della frontiera è stata scelta una scala logaritmica sull'asse y. Questa impostazione è indispensabile dato che i valori variano di diversi ordini di grandezza a seconda dell'algoritmo: nella fascia di distanza più ampia, ad esempio, Dijkstra espande 38.767 nodi, una quota oltre 44 volte superiore rispetto ad A* Pesato (880 nodi) e 68 volte superiore rispetto a Greedy (570 nodi). Su scala lineare, un divario così marcato avrebbe schiacciato i dati delle configurazioni più efficienti verso lo zero, rendendo i grafici del tutto illeggibili.

Al contrario, i pannelli del costo del percorso e del fattore di ramificazione effettivo b^* mantengono una scala lineare. Non essendoci scostamenti macroscopici tra i dati, la rappresentazione lineare si rivela la scelta migliore per apprezzare le differenze percentuali e l'effettivo andamento di convergenza delle metriche.

4.9 Validazione cartografica dei percorsi per classi di distanza

Per verificare la correttezza dei percorsi generati, il sistema esporta le mappe dei tracciati calcolati. Le prime due figure mostrano una panoramica dell'intera rete stradale della città metropolitana di Napoli, con un esempio di percorso a lungo raggio che attraversa l'area di test. Nelle figure successive sono riportati i percorsi estratti come campione per ciascuna fascia di distanza, scelti automaticamente tra i 20 test eseguiti per ogni classe. Il singolo tracciato evidenziato in rosso rappresenta il cammino minimo comune calcolato con successo dalle tre strategie ammissibili (Dijkstra, A* Euclidea e A* Landmark). Le mappe confermano visivamente il rispetto dei vincoli topologici, quali i sensi di marcia e la continuità stradale, sia nei brevi tratti urbani (0,5–1 km) sia sulle tratte extraurbane e autostradali.



Figura 1 – Mappa considerata



Figura 2 - Percorso 0.5-1 km



Figura 3 - Percorso 1-2 km

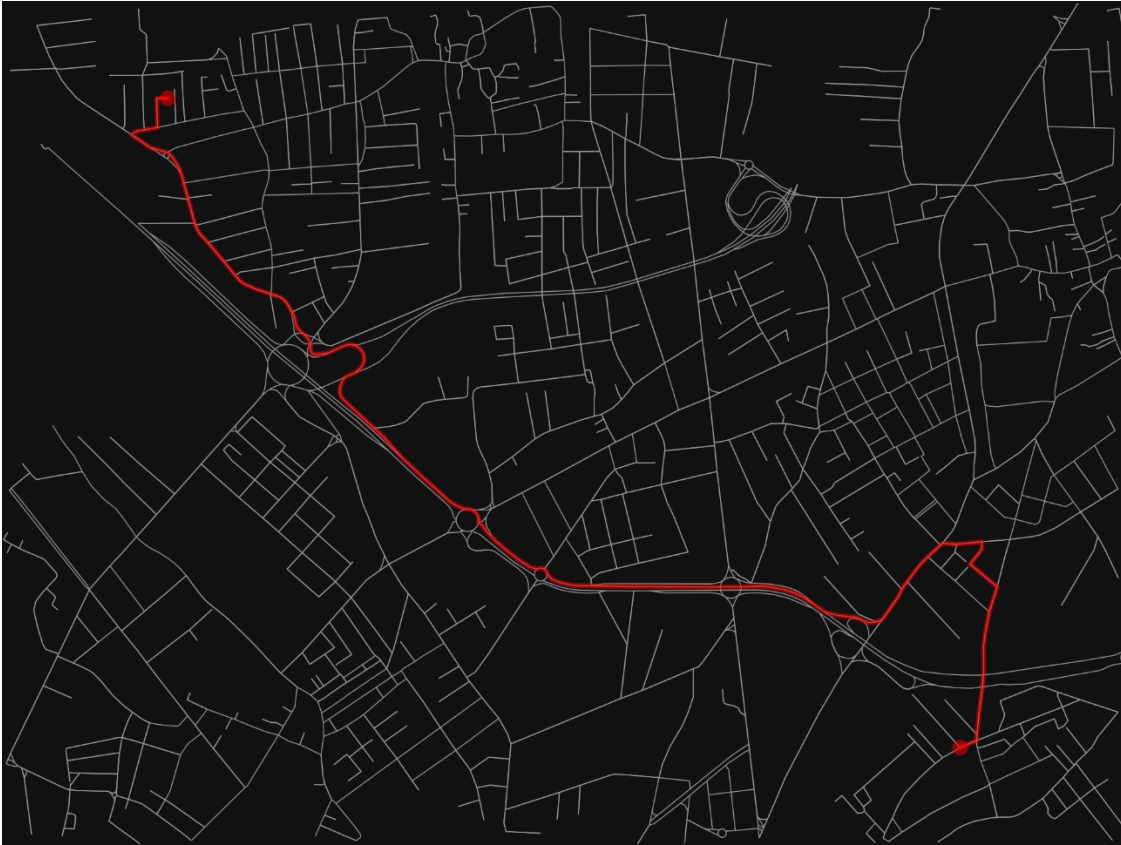


Figura 4 - Percorso 8-15 km

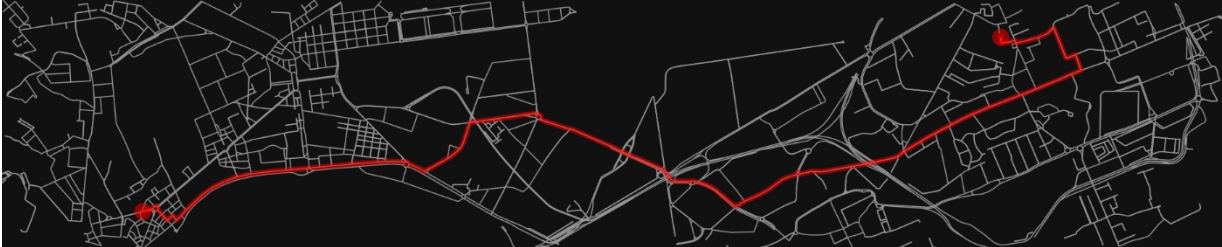


Figura 5 - Percorso 2-4 km



Figura 6 - Percorso 4-8 km

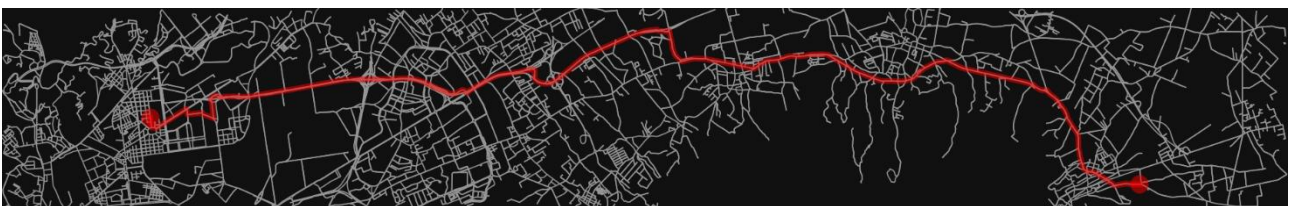


Figura 7 - Percorso 15-25 km

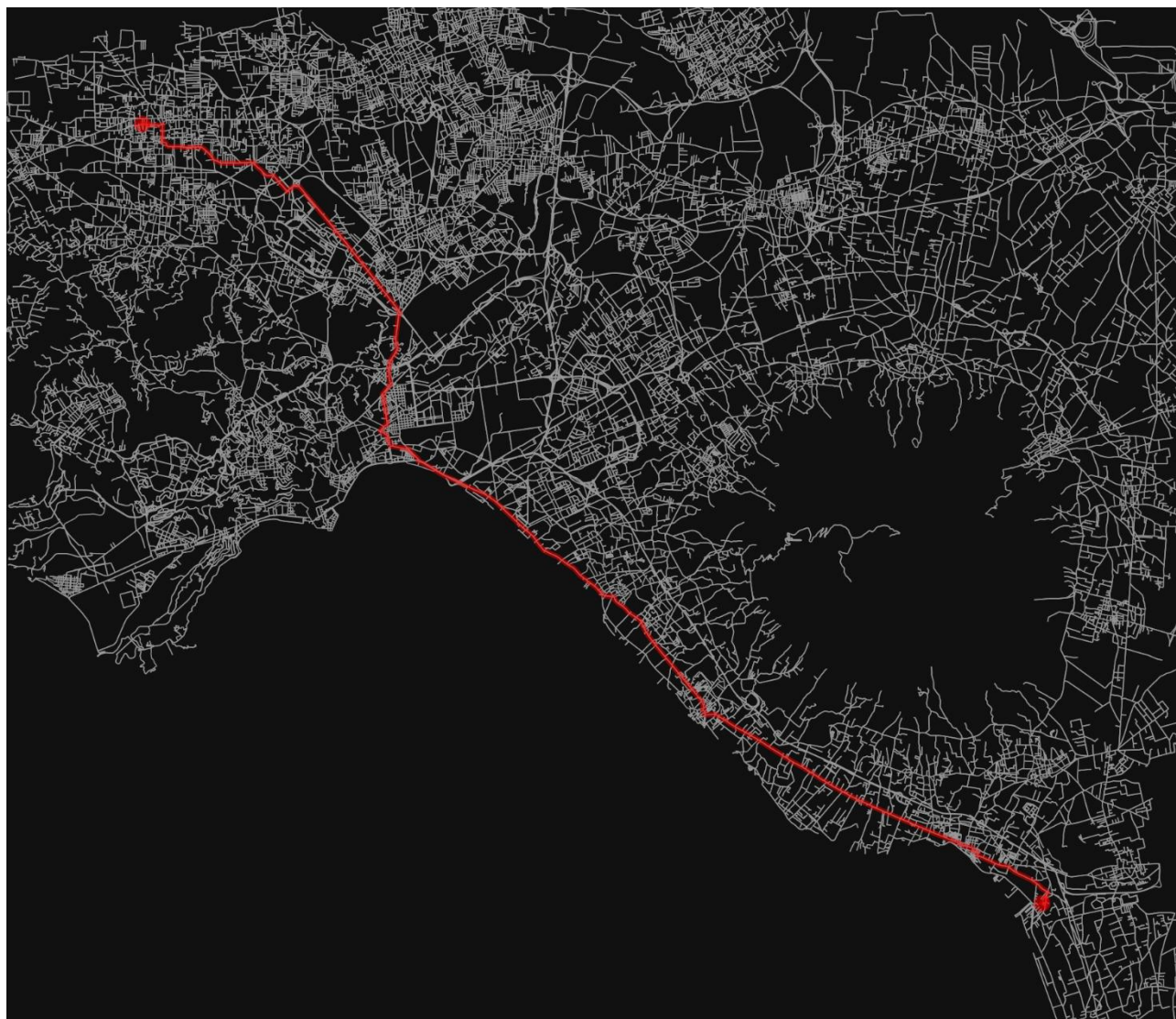


Figura 9 - Percorso 35-50 km



Figura 8 - Percorso 25-35 km

5. Conclusioni

Il progetto ha permesso di studiare il problema del routing urbano trattandolo come una ricerca offline a stato singolo in un ambiente statico, deterministico e del tutto osservabile. Utilizzando la mappa della città metropolitana di Napoli scaricata con OSMnx, è stato eseguito un benchmark automatico da 960 test totali su 8 fasce di distanza. Questo sistema ha consentito di mettere a confronto diverse strategie: le ricerche non informate (BFS e Dijkstra), l'algoritmo A* con euristiche ammissibili (Euclidea e Landmark) e le varianti non ottimali (A* Pesato e Greedy).

I dati raccolti confermano la teoria: le tre configurazioni ammissibili hanno trovato il percorso più breve in tutti i test, senza eccezioni. Guardando il numero di nodi espansi, l euristica Landmark si è rivelata sempre più efficiente della distanza Euclidea, riducendo l'area di calcolo in ogni fascia chilometrica.

I tempi di esecuzione mostrano invece che sulle distanze brevi l'approccio euclideo è più veloce perché il calcolo in linea d'aria richiede pochissime operazioni per nodo; superata la soglia dei 4 km, invece, Landmark compensa questo sovraccarico iniziale grazie a una forte riduzione dei nodi da esaminare, risultando l'opzione migliore per i percorsi a lungo raggio.

I risultati hanno infine chiarito il comportamento del fattore di ramificazione effettivo b^* . Per via della sua stessa formula matematica, questo valore tende a convergere ad 1 man mano che i percorsi diventano più lunghi e profondi. Per questo motivo b^* non può essere usato per confrontare tragitti di lunghezze molto diverse, ma resta valido solo per misurare l'efficienza di algoritmi differenti all'interno della stessa fascia di distanza.